



Fallstudie: Finalisierungsphase

Internationale Hochschule

Duales Studium

Studiengang: Informatik

Technical Softwaredesign

Jana Meyer: 10236813

Lena Pannbacker: 1240698

Dozent: Christoph Schöning

Abgabedatum: 31.09.2026

Phase 3: Finalisierungsphase

Aufbauend auf der Analyse aus der Phase 1 und dem architektonischen Entwurf aus Phase 2 wurde die Anwendung TaskMaster in dieser Phase fertiggestellt. Die geplanten Refactorings sind umgesetzt, die Anwendung ist in eine hexagonale Struktur (Domain / Application / Infrastructure) überführt und über einen Composition Root verdrahtet. Zum Nachweis der gewonnenen Erweiterbarkeit wurden zwei neue Features ergänzt, und die Qualität ist durch Unit-Tests belegt und einen Test der Kernlogik gegen gemockte Infrastruktur belegt. Das Gesamtsystem ist lauffähig (`python main.py`); alle Tests bestehen (`python -m unittest discover -s tests`). Der refaktorierte Code liegt im mitgegebenen Repository, der unrefaktorierte Ausgangsstand der Dateien liegt zur Referenz in `legacy/`.

Vorgehen:

Durchgeführt wurde ein inkrementelles Refactoring, bei dem die Lauffähigkeit der Anwendung bei jedem Schritt erhalten wurde. Jede Änderung bezog sich auf ein konkretes Problem. Die Refactoring, Aufteilung des TaskManager in Services, der Notifier-Port mit den Mustern Strategy, Composite und Decorator, sowie die Konstruktor Injektion wurden abgeschlossen und zusammengeführt. In `main.py` liegt der Composition Root, hier wird der Objektgraph aufgebaut, Repositories und Services werden dort instanziiert. Es gibt jetzt nur noch eine Database-Instanz, die von allen genutzt wird, so dass keine Inkonsistenzen mehr auftreten.

Neue Features:

Im Rahmen der Aufgabe wurden zwei neue Features in die bestehende, refaktorierte TaskMaster-Anwendung integriert, ohne die grundlegende bestehende Funktionalität zu verändern.

Es wurde eine neue Funktion `find_tasks()` entwickelt. Damit können Aufgaben nach Status, Priorität und zuständigem Benutzer gefiltert werden. Die Filterung wurde entsprechend der bestehenden Architektur im TaskRepository umgesetzt und über den TaskService bereitgestellt. Zusätzlich wurde eine Funktion `user_statistics()` implementiert. Diese ermittelt für einen Benutzer die Gesamtanzahl seiner Aufgaben sowie die Anzahl der Aufgaben in den verschiedenen Status. Außerdem werden überfällige Aufgaben ermittelt. Dabei werden erledigte und abgebrochene Aufgaben nicht als überfällig berücksichtigt.

Qualitätssicherung:

Der finale Stand umfasst 16 Unit-Tests: Tests der Domänenlogik (`test_task.py`), der Notifications (`test_notifier.py`), sowie der Kernlogik (`test_core_logic.py`). Im Test der Kernlogik wird eine gemockte Infrastuktur erzeugt. Getestet werde unter anderem das Anlegen einer Aufgabe

inklusive ausgelöster Benachrichtigung, die Ablehnung ungültiger Eingaben, der Statuswechsel sowie die korrekte Statistik- und Überfälligkeitsberechnung.

Zielerreichung:

Ziel	Status
Behebung der analysierten Probleme	Erreicht (SOLID, Kopplung/Kohäsion)
Zielarchitektur umgesetzt	Erreicht (hexagonal, composition root)
≥ 2 Entwurfsmuster	Erreicht (Strategy, Composite, Decorator, Repository)
Neue Features	Erreicht (find_tasks, user_statistics)
Kernlogik-Test	Erreicht (test_core_logic.py)
Lauffähiges Gesamtsystem	Erreicht

Reflexion:

Der inkrementelle, prinzipiengetriebene Ansatz hat sich bewährt. Die lauffähige Zwischenversion nach jedem Schritt hielt das Risiko gering, größere neue Probleme im Code zu erzeugen. Die Aufteilung entlang der Ports ermöglichte es, parallel am Code zu arbeiten. Dass sich beide Features ergänzen ließen, ohne bestehende Klassen zu brechen, bestätigte den Vorteil der gewählten Architektur. Insgesamt zeigt das Projekt, dass Wartbarkeit und Erweiterbarkeit durch die Definition klarer Schnittstellen deutlich erhöht werden.